Detección Automática de Crisis Epilépticas en Señales EEG mediante una Red Neuronal Convolucional 1D (CNN-1D)

Dataset CHB-MIT Scalp EEG Database

Agostina Pérez - Inteligencia Artificial

2026

1. Introducción y definición del problema

La inspección visual de registros de electroencefalograma (EEG) es un proceso complejo y demandante para los neurólogos debido a la magnitud de los datos que produce un monitoreo continuo de largo plazo. El objetivo de este proyecto es implementar una Red Neuronal Convolucional unidimensional (CNN-1D), capaz de clasificar ventanas temporales de señal cruda de EEG para diferenciar automáticamente entre períodos de crisis (actividad ictal) y períodos de actividad normal (basal).

El sistema opera de forma offline sobre registros pregrabados en formato EDF. Para cada ventana temporal de la señal, el modelo emite una probabilidad de crisis que, comparada contra un umbral de decisión, se traduce en una clasificación binaria de crisis o no crisis. El desempeño se evalúa con métricas clínicas de clasificación binaria y, de forma complementaria, con métricas a nivel de evento (crisis completa), que son las que resultan relevantes para la práctica médica.

El alcance de la primera entrega (MVP) consiste en un conjunto de scripts por línea de comandos que cubren el ciclo completo: partición del dataset, preprocesamiento, entrenamiento, evaluación, selección de modelo e inferencia sobre un archivo nuevo.

2. Formulación del agente (framework PEAS)

Para especificar formalmente el agente se utiliza el framework PEAS (Performance, Environment, Actuators, Sensors).

Componente	Descripción
Performance (P)	Clasificar correctamente cada ventana temporal de EEG como crisis (ictal) o no crisis (basal). Se cuantifica con sensibilidad (recall), especificidad, tasa de falsas alarmas (FPR) y falsos positivos por hora. El objetivo es maximizar la sensibilidad manteniendo el FPR y los falsos positivos por hora lo más bajos posible.
Environment (E)	El contexto clínico de monitoreo EEG continuo: actividad eléctrica cerebral del paciente captada en 16 canales bipolares (montaje TUEV) a 256 Hz, junto con ruido y artefactos (musculares, oculares, de movimiento, interferencia de red). La señal es continua y altamente no estacionaria.
Actuators (A)	El agente no actúa físicamente sobre el paciente; es un sistema de apoyo a la decisión. Su acción es emitir la clasificación binaria (crisis / no crisis) de cada ventana y, en un despliegue real, generar una alarma o aviso al personal médico.
Sensors (S)	Los electrodos del montaje, que miden diferencias de potencial sobre el cuero cabelludo. La señal cruda se digitaliza a 256 Hz, se filtra en banda 0.5 a

	50 Hz y se segmenta en ventanas de 5.12 s (1310 muestras, duración efectiva 5.117 s, con 50% de solapamiento).

2.1 Características del entorno

- Observabilidad: parcialmente observable. La señal útil está contaminada por ruido y artefactos; la percepción es una ventana de señal, no el estado completo del cerebro.
- Determinismo: estocástico. La señal de EEG es aleatoria y no estacionaria; además el modelo introduce aleatoriedad (inicialización de pesos, dropout, lotes estocásticos) y su salida es una probabilidad.
- Secuencialidad: episódico. Cada ventana se clasifica de forma independiente; la decisión sobre una ventana no condiciona las siguientes.
- Dinamismo: estático. En el momento de la clasificación el agente opera sobre una ventana de señal congelada y su acción no modifica el entorno.
- Continuidad: continuo en la entrada (amplitudes reales) y discreto en la salida (crisis / no crisis), discretizada por un umbral de decisión.
- Número de agentes: agente individual, que clasifica de forma autónoma.

2.2 Tipo de programa de agente

El agente aprende mediante aprendizaje supervisado. Su arquitectura interna se compone de: elemento de actuación (la CNN-1D entrenada, SeizureCNN), crítica (la función de pérdida durante el entrenamiento y las métricas clínicas en validación y test), elemento de aprendizaje (el optimizador AdamW con retropropagación) y generador de problemas (el muestreo balanceado y aleatorizado del entrenamiento). En inferencia, el agente se comporta como un agente basado en utilidad: la salida sigmoide es una utilidad continua (probabilidad de crisis) que se compara contra un umbral configurable.

3. Datos: CHB-MIT Scalp EEG Database

Se utiliza un dataset clínico de dominio público del Hospital Infantil de Boston (CHB) en colaboración con el MIT, denominado CHB-MIT Scalp EEG Database. El contenido aproximado es de 24 pacientes (chb01 a chb24), cerca de 900 archivos EDF de aproximadamente una hora cada uno y unos 43 GB en total. La frecuencia de muestreo es de 256 Hz.

Cada archivo trae del orden de 23 canales, aunque el montaje varía entre pacientes: algunos pacientes tienen submontajes y otros usan un montaje distinto. El proyecto trabaja con un montaje canónico de 16 canales bipolares compatible con el montaje del dataset TUEV, para asegurar la compatibilidad a la hora de realizar el Transfer Learning (implementación futura)

3.1 Anotaciones clínicas

Cada paciente tiene un archivo de resumen (chbXX-summary.txt) que lista, para cada EDF, los intervalos de crisis con su tiempo de inicio (Seizure Start Time) y de fin (Seizure End Time) en segundos. El módulo src/annotations.py parsea estos archivos, tolerando dos formatos distintos según el paciente (con o sin numeración de crisis), y los convierte en una estructura paciente -> archivo -> lista de intervalos (inicio, fin).

4. Etapa 1: Split inter-paciente

El split se realiza por paciente, evitando así la fuga de datos (data leakage) ya que ventanas del mismo paciente están altamente correlacionadas (solapamiento temporal y características propias del sujeto).

Los pacientes se ordenan por severidad descendente (segundos de crisis, con desempate por cantidad de archivos). Luego se recorre en ese orden y, para cada paciente, se evalúan las dos opciones (enviarlo a train o a test) eligiendo la que deja el acumulado global más cerca del objetivo. El volumen se mide combinando dos métricas: fracción de archivos (peso 1.0) y fracción de segundos de crisis (peso SPLIT_W_ZSEC = 2.0). Ponderar el doble los segundos de crisis busca equilibrar la cantidad de ventanas positivas en test.

La configuración permite fijar pacientes a un conjunto concreto para garantizar la presencia de casos difíciles (crisis cortas) en cada partición. En src/config.py se definen:

- VAL_REQUIRED_PATIENTS = ['chb14'] -> paciente obligatorio de validación (crisis cortas)
- FORCED_TRAIN_PATIENTS = ['chb06'] -> paciente forzado a entrenamiento
- FORCED_TEST_PATIENTS = ['chb16'] -> paciente forzado a test (caso difícil, crisis de ~8 s)

De la porción de train se reservan 5 pacientes para validación. La selección de validación toma pacientes del medio en severidad (evitando extremos) y garantiza la inclusión de los pacientes obligatorios.

El resultado se persiste en data/processed/split.json con las listas de pacientes por conjunto, el detalle por paciente (severidad) y un resumen de volúmenes para verificar la proporcionalidad. El diccionario incluye un identificador estable split_id calculado como hash de los campos reproducibles, que luego se usa para validar la compatibilidad entre split, escalador y checkpoints.

5. Etapa 2: Preprocesamiento de la señal (src/preprocessing.py, src/annotations.py)

Cada EDF se lee con la librería MNE. Se normalizan los nombres de canal y se resuelve un plan de selección para obtener los 16 canales del montaje TUEV en orden canónico. El plan soporta tres casos: canal bipolar directo, resta de dos electrodos únicos, y reconstrucción por resta de dos canales que comparten una referencia común.

Se aplica un filtro Butterworth pasabanda de 0.5 a 50 Hz de orden 5, construido en formato SOS (secciones de segundo orden) para estabilidad numérica. Se usa sosfiltfilt, que filtra hacia adelante y hacia atrás, anulando el desfase temporal (fase cero) a costa de no ser causal y de duplicar la respuesta en magnitud (orden efectivo 10).

La señal se segmenta en ventanas deslizantes. Los parámetros de segmentación son:

- WIN_SECONDS = 5.12 s (duración nominal)
- WIN_SAMPLES = 1310 (5.12 * 256 redondeado hacia abajo, número par)
- WIN_SECONDS_EFFECTIVE = 5.1171875 s
- STRIDE_SAMPLES = 655 (50 % de solapamiento)

La cantidad de ventanas de una señal de N muestras se calcula como (N - WIN_SAMPLES) // STRIDE_SAMPLES + 1, descartando la sobra final para emitir solo ventanas completas.

Una ventana se etiqueta como crisis (1) si su intervalo de tiempo solapa cualquier porción de un intervalo de crisis anotado; en caso contrario se etiqueta como no crisis (0). El solapamiento se evalúa de forma vectorizada con máscaras booleanas.

Cada canal se centra en su mediana y se escala por su rango intercuartílico (IQR = Q75 - Q25), siguiendo la transformación x' = (x - mediana_c) / IQR_c. Se usa el RobustScaler de scikit-learn con quantile_range (25, 75). Las estadísticas (un par mediana/IQR por canal) se estiman exclusivamente sobre las ventanas de train y se reutilizan tal cual en validación, test e inferencia, evitando la fuga de datos. La mediana y el IQR son estadísticos de orden robustos a valores atípicos (artefactos), a diferencia de media/desvío (z-score) o mínimo/máximo (min-max).

Para calcular mediana e IQR utilizo un submuestreo uniforme (ya que procesar todo sería inviable por memoria): una de cada 16 ventanas (STATS_WINDOW_STRIDE) y, dentro de cada una, una de cada 16 muestras (STATS_SAMPLE_STRIDE), con un generador aleatorio de semilla fija. Esto deja aproximadamente 55 millones de valores (unos 220 MB). En valores robustos, un submuestreo uniforme converge al valor del train completo.

Las estadísticas se guardan en data/processed/scaler_stats.npz junto con metadata de auditoría: cantidad de archivos, ventanas y puntos usados, versión del protocolo, split_id y scaler_id (hash de canales y estadísticas) y la configuración de preprocesamiento. Estos identificadores permiten detectar incompatibilidades si el split o el preprocesamiento cambian entre corridas.

6. Etapa 3: Dataset y DataLoader (src/data.py)

La clase WindowDataset (hereda Dataset de PyTorch) construye una sola vez un índice global ventana -> (archivo, ventana local, etiqueta) leyendo solamente las cabeceras de los EDF, sin cargar la señal completa. La señal de cada archivo se procesa recién cuando hace falta y se conserva en RAM solo el último archivo procesado (caché de un archivo), porque process_edf devuelve todas las ventanas de un archivo de una vez.

La crisis representa aproximadamente el 1 por ciento del registro. Para no aprender el clasificador trivial (predecir siempre no crisis) se combinan dos mecanismos durante el entrenamiento: submuestreo de la clase mayoritaria y pérdida ponderada. Validación y test se evalúan siempre sobre la distribución natural completa, sin tocar.

Cada época de entrenamiento se arma en cinco pasos: (1) se eligen todas las ventanas positivas más NEG_POS_RATIO veces esa cantidad de negativas elegidas al azar sin repetición; (2) se agrupan esas ventanas por archivo; (3) cada archivo se procesa una única vez; (4) todas las ventanas elegidas se concatenan y se mezclan completamente en RAM; (5) se entregan de a una y el DataLoader arma los lotes. La semilla es reproducible y el generador de ruido usa una semilla distinta.

Durante el entrenamiento se suma ruido gaussiano N(0, NOISE_STD) exclusivamente a las ventanas positivas (clase minoritaria). El desvío NOISE_STD = 0.1 está en unidades de la señal ya escalada (donde la mediana es cercana a 0 y el IQR a 1). El objetivo es actuar como regularización: en cada época el modelo ve una versión levemente distinta de la misma ventana de crisis, lo que favorece la generalización. El generador de ruido usa una semilla independiente para no mezclarse con el azar del submuestreo.

La función build_splits_dataloaders devuelve los tres conjuntos: train usa el BalancedEpochDataset (balanceado y mezclado, con num_workers forzado a 0 para no duplicar la época), mientras que val y test recorren todas las ventanas en orden natural sin mezclar. Se soporta un límite de archivos (limit_files) para pruebas rápidas (smoke tests).

7. Etapa 4: Arquitectura de la red (src/model.py)

La red SeizureCNN es una CNN-1D apilada formada únicamente por bloques Conv1d + BatchNorm1d + ReLU + MaxPool1d, seguidos de Global Average Pooling, dropout y un clasificador denso. No usa BiLSTM, atención ni representaciones 2D.

Entrada: (batch, 16 canales, 1310 muestras)

Bloque 1: Conv1d(16 -> 64, k=7) + BatchNorm + ReLU + MaxPool(2)
Bloque 2: Conv1d(64 -> 128, k=5) + BatchNorm + ReLU + MaxPool(2)
Bloque 3: Conv1d(128 -> 256,k=5) + BatchNorm + ReLU + MaxPool(2)
Bloque 4: Conv1d(256 -> 256,k=3) + BatchNorm + ReLU + MaxPool(2)
Bloque 5: Conv1d(256 -> 256,k=3) + BatchNorm + ReLU + MaxPool(2)
Bloque 6: Conv1d(256 -> 256,k=3) + BatchNorm + ReLU + MaxPool(2)

Global Average Pooling (AdaptiveAvgPool1d(1))
Dropout(0.4)
Linear(256 -> 256) + ReLU + Dropout(0.4)
Linear(256 -> 1) -> logit
Sigmoid (en predict_proba) -> probabilidad de crisis

- Conv1d usa padding = kernel // 2 (padding 'same') para conservar la longitud temporal dentro del bloque.
- BatchNorm1d normaliza las activaciones por canal (media 0, varianza 1) y luego reescala con dos parámetros aprendidos por canal (gamma y beta).
- ReLU introduce la no linealidad entre capas.

- MaxPool1d(2) reduce la longitud temporal a la mitad, quedándose con el máximo de cada par de muestras.
- Global Average Pooling promedia cada canal a lo largo del tiempo, produciendo un vector de 256 características que resume cuánto se activó cada filtro.
-

La forma de los tensores a lo largo del flujo de la red es:

```
(B, 16, 1310) --> 6 bloques-->  (B, 256, 20) -> GAP -> (B, 256, 1) -> squeeze -> (B,
        256)-> Dropout -> FC(256->256)+ReLU+Dropout -> FC(256->1) -> (B, 1)
```

El modelo tiene aproximadamente de 0.5 a 2 millones de parámetros entrenables. El recuento exacto y el desglose por bloque (convolucional frente a densa) se calcula en src/compute_metrics.py.

8. Etapa 5: Entrenamiento (src/train.py)

Para la optimización se usa AdamW, que combina el optimizador Adam (momentum y tasa de aprendizaje adaptativa por parámetro) con weight decay desacoplado: en cada paso se encoge cada peso hacia cero de forma directa, sin mezclarse con el momento adaptativo. Esto actúa como regularización L2 contra el sobreajuste. Los valores por defecto son learning rate 1e-4 y weight decay 1e-2.

8.1 Función de pérdida BCE con pos_weight

Para el escenario baseline se usa BCEWithLogitsLoss con pos_weight. La pérdida binaria aplica internamente la sigmoide al logit y mide el error con la fórmula L = -[y*log(p) + (1-y)*log(1-p)]. El pos_weight multiplica el error de las ventanas de crisis para contrarrestar el desbalance. La pérdida de validación se calcula siempre sin pos_weight, para que val_loss sea comparable entre escenarios.

La función compute_positive_weight devuelve el peso a aplicar a la clase crisis en la pérdida binaria. Se calcula como el cociente entre la cantidad real de negativas muestreadas y la cantidad de positivas, de modo que refleje la distribución efectivamente muestreada. Con NEG_POS_RATIO = 3, cada ventana de crisis pesa aproximadamente 3 veces una ventana normal en la pérdida.

8.2 Focal Loss

Se implementó una clase FocalLoss que opera sobre logits crudos. La fórmula es:

```
FL(p_t) = -alpha_t * (1 - p_t)^gamma * log(p_t)
```

donde p_t es la probabilidad asignada a la clase correcta, alpha_t pondera la clase rara (crisis) y gamma regula cuánto se desincentivan los ejemplos bien clasificados (fáciles). Con gamma = 2.0 y alpha = 0.25 el gradiente se concentra en los ejemplos difíciles (crisis cortas y falsos positivos) en lugar de dejar que la masa de ventanas normales fáciles domine el entrenamiento. El alpha es el factor de la clase positiva: con 0.25 las positivas pesan 0.25 y las negativas 0.75.

La elección de pérdida se controla por línea de comandos con --loss bce o --loss focal, junto con --focal-gamma y --focal-alpha. El tipo de pérdida y sus parámetros se guardan en el checkpoint y en el historial para auditoría.

8.4 Loop de entrenamiento y evaluación

train_one_epoch ejecuta, por lote: movimiento al dispositivo, puesta a cero de gradientes, paso hacia adelante, cálculo de la pérdida, retropropagación (backward) y paso del optimizador. Devuelve la pérdida promedio ponderada por tamaño de lote. La función evaluate recorre el DataLoader en modo evaluación (sin dropout y con estadísticas acumuladas de BatchNorm) y devuelve val_loss, todas las probabilidades (sigmoide) y todas las etiquetas.

8.5 Early stopping

El val_loss controla la paciencia (PATIENCE = 8): si no mejora en ese número de épocas, se detiene el entrenamiento. El early stopping es independiente de la elección del checkpoint, de modo que una época que no mejora el loss aún puede ser el mejor checkpoint clínico.

8.6 Selección del checkpoint (criterio clínico)

El checkpoint no se elige por el mínimo del loss, sino por el criterio clínico de validación. En cada época se calcula el punto de operación a nivel de evento: entre los umbrales cuya mediana de falsas alarmas por paciente no supera MAX_FALSE_ALARMS_PER_HOUR (1 por hora), se elige el de mayor sensibilidad de crisis, desempatando por menor falsa alarma y luego por AUPRC. Si ninguna época cumple el techo, se usa val_loss como fallback y la situación queda registrada. El checkpoint guarda el umbral operativo (op_threshold), sus métricas (op_metrics) y la bandera event_feasible.

8.7 Historial y reporte

Además del checkpoint, se guarda un archivo <nombre>.history.json con las métricas por época (pérdidas, métricas de ventana y evento, AUPRC, AUC-ROC, tiempos) y un reporte de cómputo, para generar las curvas de entrenamiento sin reentrenar.

9. Etapa 6: Métricas de evaluación (src/metrics.py)

9.1 Nivel ventana (segmento)

Sobre la matriz de confusión (TP, FP, TN, FN) se calculan, para un umbral dado:

- Sensibilidad (recall) = TP / (TP + FN)
- Especificidad = TN / (TN + FP)
- FPR (tasa de falsos positivos) = FP / (FP + TN) = 1 - Especificidad
- Falsos positivos por hora = FP / horas cubiertas
- Accuracy = (TP + TN) / total
- F1 = 2 * precision * recall / (precision + recall)

- MCC = (TP*TN - FP*FN) / sqrt((TP+FP)(TP+FN)(TN+FP)(TN+FN))

El MCC (coeficiente de correlación de Matthews) es robusto al desbalance de clases, porque considera las cuatro celdas de la confusión. Las horas cubiertas se calculan considerando el solapamiento del 50 por ciento entre ventanas.

9.2 Métricas independientes del umbral

La función rank_metrics calcula AUC-ROC y AUPRC (average precision) sobre las probabilidades y las etiquetas, sin depender del umbral:

- AUC-ROC: la capacidad que tiene un modelo de clasificación binaria para distinguir entre dos clases diferentes (roc_auc_score).
- AUPRC: área bajo la curva precisión-recall (average_precision_score), recomendada bajo desbalance extremo porque el AUROC tiende a inflarse con desbalance.

9.3 Nivel evento y postprocesado

Se dispara una alarma cuando, en las últimas N ventanas consecutivas, hay al menos n predicciones positivas. Con N = 3 y n = 2 (configuración por defecto), la alarma exige evidencia repetida de crisis y filtra falsos positivos aislados (artefactos en una única ventana). Los valores están en config.py como WINDOW_RANGE_FOR_EVENT = 3 y POSITIVES_FOR_EVENT = 2.

Después de disparar una alarma se ignoran nuevas alarmas durante un lapso de 240 segundos (4 minutos), para que una crisis larga no genere múltiples alarmas que se contarían como falsas alarmas extra.

Una crisis se considera detectada si hay una alarma en el intervalo [onset, onset + max_latency], con la latencia máxima siendo 30 segundos. La latencia de detección es alarma - onset. Si no hay alarma en ese intervalo, la crisis se cuenta como no detectada (falso negativo de evento).

Las métricas a nivel evento son

- Sensibilidad de evento = crisis detectadas / crisis totales
- FDR (falsas alarmas por hora) = falsas alarmas / horas de registro, reportado también como mediana por paciente
- Latencia media y mediana
- Conteos: n_seizures, n_detected, n_early_detections, n_false_alarms, total_hours

9.4 Selección del punto de operación y fallback

select_event_operating_point barre una malla de umbrales (THRESHOLD_GRID de 0.05 a 0.95 en pasos de 0.05) y, entre los que respetan el techo de falsas alarmas, elige el de mayor sensibilidad de evento, con desempate por menor mediana de falsas alarmas. Si ningún umbral cumple el techo, select_lowest_false_alarm_point devuelve el de menor mediana de falsas alarmas como fallback explícito, marcado como no clínicamente factible.

9.5 Agrupación de eventos para inferencia

detect_event_intervals agrupa ventanas positivas consecutivas en episodios de crisis para la inferencia sin anotaciones: el inicio es la primera ventana positiva y el fin la última, fusionando episodios separados por menos del intervalo mínimo entre alarmas.

10. Etapa 7: Evaluación sobre val/test (src/evaluate.py)

src/evaluate.py carga un checkpoint, reconstruye la CNN, arma el DataLoader del split solicitado (val o test) sin balancear y sin pos_weight, y calcula las métricas clínicas. Valida que el checkpoint contenga los metadatos obligatorios y que el split_id y el escalador coincidan con el split actual.

En test, el umbral usado es el op_threshold seleccionado durante la validación (no se recalibra sobre test). En validación, el punto de operación se vuelve a seleccionar con la regla de evento. El informe imprime las métricas de ranking (AUC-ROC y AUPRC), las métricas de ventana (sensibilidad, especificidad, FPR, falsos positivos por hora, accuracy, F1, MCC y conteos) y las métricas de evento (sensibilidad, falsas alarmas por hora globales y mediana por paciente, latencia y desglose por paciente).

11. Etapa 8: Inferencia sobre un EDF (src/inference.py)

src/inference.py procesa un EDF sin necesidad de anotaciones: aplica el mismo filtro, ventaneo y escalador guardado en el checkpoint, y produce una predicción por ventana. La decisión temporal se considera disponible al final de cada ventana. El postprocesado agrupa las ventanas positivas en episodios de crisis mediante detect_event_intervals. El informe muestra en pantalla los eventos detectados y guarda en disco el detalle ventana a ventana en un archivo JSON (directorio inferences/ por defecto).

12. Resultados

Los resultados se reportan en dos niveles complementarios. Primero, la validación cruzada agrupada por paciente compara las tres configuraciones de función de pérdida sobre el pool de desarrollo (18 pacientes) para seleccionar la ganadora. Segundo, la ganadora se reentrena sobre todo el pool de desarrollo y se evalúa una sola vez sobre el test congelado inter-paciente (6 pacientes, 34 crisis), comparando contra los baselines clásicos.

12.1 Validación cruzada de desarrollo

La validación cruzada usa StratifiedGroupKFold con K = 5 folds y estratificación por severidad de crisis (segundos de crisis por paciente), con semilla fija 42. La unidad de agrupación es el paciente, de modo que ninguna ventana del mismo sujeto comparte entrenamiento y validación. En cada fold se entrenan las tres configuraciones (bce, focal_a025 y focal_a075) con los pacientes de los K−1 folds restantes y se evalúa con el fold excluido; cada paciente cae exactamente una vez en validación. Las estadísticas del RobustScaler se recalculan en cada fold únicamente con sus pacientes de entrenamiento, evitando fuga de datos.

Dado que las métricas por fold presentan una distribución fuertemente sesgada (algunos folds contienen pocas crisis y producen valores atípicos), los resultados se resumen con la mediana y el rango intercuartílico (IQR), más robustos que la media ± desvío. La sensibilidad de evento se reporta como valor pooled, sumando crisis detectadas y totales a través de los folds, lo que pondera por cantidad de crisis y no por fold.

Configuración	AUPRC (mediana, IQR)	AUC-ROC (mediana, IQR)	Sens. evento (pooled)	FA/h mediana (mediana, IQR)
bce	0,118 (0,011–0,271)	0,842 (0,822–0,918)	48,2 % (79/164)	0,787 (0,474–0,922)
focal_a025	0,138 (0,037–0,318)	0,874 (0,838–0,916)	55,5 % (91/164)	0,725 (0,642–0,798)
focal_a075	0,093 (0,018–0,326)	0,874 (0,840–0,928)	50,0 % (82/164)	0,713 (0,410–0,756)

La configuración ganadora se selecciona por la mediana de AUPRC de validación, con desempate por menor IQR. Bajo este criterio resulta ganadora focal_a025 (mediana 0,138), seguida de bce (0,118) y focal_a075 (0,093). La ganadora se reentrena sobre los 18 pacientes del pool de desarrollo con 9 épocas (media de las épocas óptimas por fold) y se evalúa una vez sobre test con umbral fijo 0,42 (media de los umbrales seleccionados por fold).

12.2 Evaluación sobre test y comparación contra baselines

El refit de focal_a025 se evalúa una sola vez sobre el test congelado inter-paciente, sin recalibrar el umbral. Se compara contra los dos baselines clásicos (regresión logística y SVM lineal sobre 193 features de potencia por banda) y contra las configuraciones entrenadas con el split fijo. «Final» corresponde al modelo entrenado con el split train/val y punto de operación elegido en validación; «refit CV» es la ganadora de la validación cruzada reentrenada sobre el pool completo.

Métrica	Regresión logística	SVM lineal	CNN · BCE	CNN · Focal α=0,25 (final)	CNN · Focal α=0,25 (refit CV)
AUC-ROC	0,694	0,710	0,8915	0,8957	0,8606
AUPRC	0,030	0,064	0,3736	0,3881	0,3994
Sensibilidad de crisis (evento)	26,5 % (9/34)	17,6 % (6/34)	64,7 % (22/34)	67,6 % (23/34)	55,9 % (19/34)
Falsas alarmas/h (mediana por paciente)	12,55	0,41	2,05	2,42	0,61
Sensibilidad (ventana)	0,593	0,189	67,3 %	63,6 %	50,1 %
Especificidad (ventana)	69,6 %	99,6 %	95,7 %	97,2 %	99,4 %

MCC (ventana)	—	—	0,139	0,164	0,267
Latencia media de detección	14,9 s	17,6 s	16,70 s	15,85 s	16,32 s

El refit CV respeta por primera vez el techo de 1 falsa alarma/h (mediana 0,61), aunque a costa de una menor sensibilidad de crisis: 55,9 % frente al 67,6 % del modelo final previo, que en test disparaba 2,42 falsas alarmas/h. El ranking se mantiene robusto: el refit alcanza el mayor AUPRC (0,399) y un AUC-ROC de 0,861, muy por encima de los baselines (0,71 y 0,06 de AUC-ROC; 0,030 y 0,064 de AUPRC).

Paciente	Crisis	Detectadas	Sensibilidad	FA/h
chb16	10	2	0,200	1,53
chb07	3	2	0,667	0,60
chb18	6	2	0,333	0,25
chb20	8	8	1,000	0,14
chb17	3	1	0,333	0,62
chb09	4	4	1,000	1,59

El desglose por paciente revela una variabilidad extrema: chb20 (8/8) y chb09 (4/4) se detectan por completo, mientras que chb16 (2/10), chb18 (2/6) y chb17 (1/3) quedan muy por debajo. chb09 es el único paciente que excede el techo de falsas alarmas (1,59/h) pese a detectar todas sus crisis.

12.3 Análisis crítico

- El refit CV es el primer modelo en respetar el techo de 1 falsa alarma/h (mediana 0,61), pero su sensibilidad de crisis (55,9 %) queda lejos del objetivo exploratorio de ≥ 90 %.
- El compromiso entre sensibilidad y falsas alarmas depende del umbral fijo: 0,42 (media de los umbrales por fold) es más conservador que el punto de operación del modelo final previo.
- Las crisis cortas siguen siendo el punto débil (chb16, ~8 s: 2 de 10 detectadas), un problema de representación de datos más que de la función de pérdida.
- Con 34 crisis de test, las diferencias entre configuraciones tienen alta incertidumbre estadística.

13. Métricas de cómputo (src/compute_metrics.py)

Para permitir una comparación futura justa entre la CNN-1D y una CNN-2D, se miden y guardan métricas de cómputo separadas en dos familias: independientes del hardware y dependientes del hardware.

- count_parameters: parámetros totales, entrenables y desglose por bloque (convolucional, denso, otros).
- measure_flops: MACs y FLOPs por ventana (FLOPs aproxima 2 veces MACs) mediante torchinfo.
- peak_ram_mb: memoria RAM pico del proceso (psutil).
- measure_inference_latency: latencia media y p95 por ventana (con warm-up) y segundos por hora de EEG.

El reporte de cómputo (hardware, parámetros, FLOPs, RAM/VRAM pico, latencia y tiempos de entrenamiento) se vuelca en el archivo .history.json generado por el entrenamiento.

14. Referencias

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. Proceedings of the IEEE, 86(11), 2278-2324.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.

He, H., & Garcia, E. A. (2009). Learning from imbalanced data. IEEE Transactions on Knowledge and Data Engineering, 21(9), 1263-1284.

Ioffe, S., & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. arXiv:1502.03167.

LeCun, Y., Bottou, L., Orr, G. B., & Muller, K.-R. (1998). Efficient BackProp. En Neural Networks: Tricks of the Trade (pp. 9-50). Springer.

Lin, T.-Y., Goyal, P., Girshick, R., He, K., & Dollar, P. (2017). Focal loss for dense object detection. Proceedings of the IEEE International Conference on Computer Vision (ICCV), 2980-2988.

Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. PLOS ONE, 10(3), e0118432.

Wang, X., Wang, G., Wu, T., Wang, Y., Karkkainen, T., & Cong, F. (2025). EEG-based seizure onset detection of frontal and temporal lobe epilepsies using 1DCNN. IEEE Transactions on Neural Systems and Rehabilitation Engineering, 33, 2263-2272.

Schirrmeister, R. T., et al. (2017). Deep learning with convolutional neural networks for EEG decoding and visualization. Human Brain Mapping (arXiv:1703.05051).

Roy, Y., et al. (2019). Deep learning-based electroencephalography analysis: a systematic review. arXiv:1901.05498.